FlowBuilder: how it works today

A multi-tenant platform for building and running LLM workflows (upload and chat pipelines) on a visual canvas. This note covers what exists on `main` as of 13 Nov 2026: the services, the request path, the data model, and the file each piece lives in.

Source: commit 5f1838b. Build plan phase P01 (Foundation), subphases S01–S04.

BUILT

Docker stack, CI, observability, full database schema with row-level security, signup / login / logout / sessions, health checks.

SCAFFOLD ONLY

Frontend is a single placeholder page. Celery worker runs but has no tasks. Workflow, connector and run tables exist with no API yet.

DESIGNED, NOT BUILT

The workflow runner. A spike proved the design and froze it in `docs/runner-contract.md`.

1. The running system

Everything runs from one Compose file, `infra/docker-compose.yml`. Ports bind to 127.0.0.1 only.

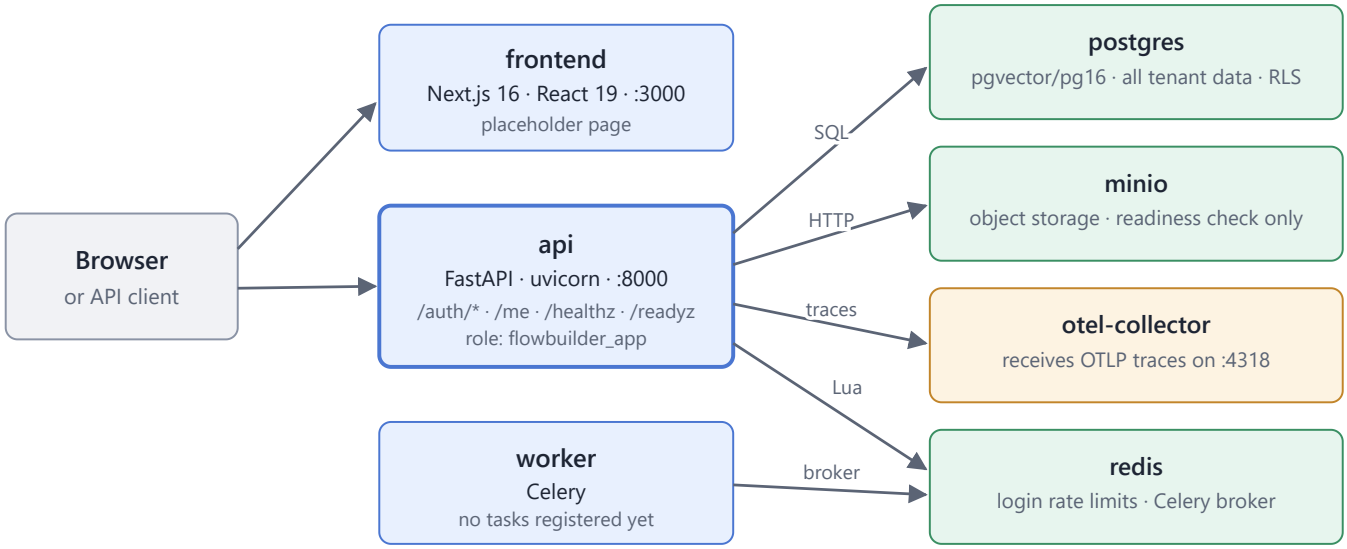


Figure 1. Runtime topology. Blue: built from this repo. Green: stock images for data. Amber: observability.

Startup order

Each process connects with only the database role it needs. Compose enforces the order with health and completion conditions.

1. postgres
healthy
Waits for `pg_isready`.

→

2. db-bootstrap
as superuser · one-shot
Creates extensions `vector`, `citext` and roles `flowbuilder_owner`, `flowbuilder_app`.

→

3. migrate
as owner · one-shot
`alembic upgrade head`, then creates run partitions 12 months ahead.

→

4. api
as app role
Refuses to start if its role could bypass row-level security. Then serves.

The api also waits for redis, minio and otel-collector to be healthy. The worker only needs redis.

2. How a request is handled

The API is built by an app factory, `create_app()` in `main.py`, which receives every dependency explicitly. Each request passes through the same layers.

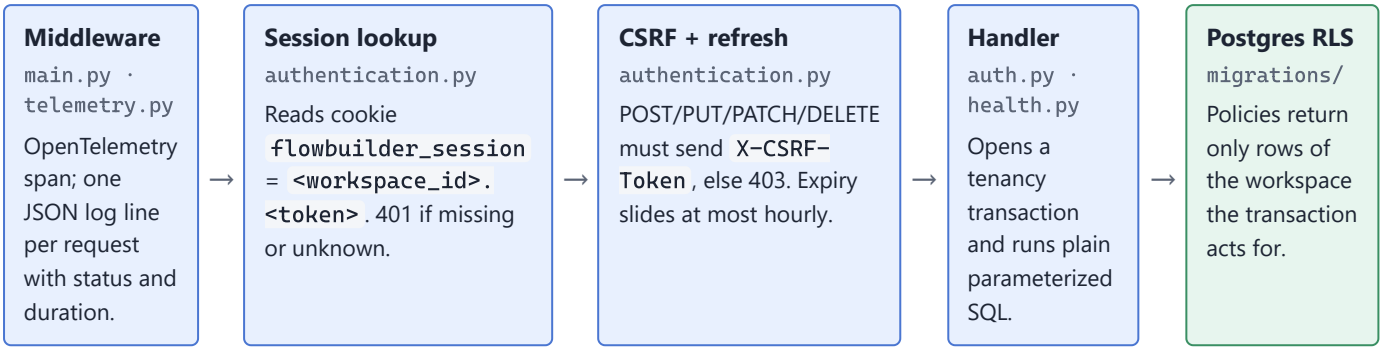


Figure 2. Request path for an authenticated route. Health routes skip the session steps.

Tenant isolation: the core idea

Isolation is enforced by Postgres, not by remembering a `WHERE workspace_id = …` in every query. Every tenant table has `FORCE ROW LEVEL SECURITY` and one policy:

```
USING (workspace_id = app_current_workspace_id())
```

`tenancy.py` is the only place that sets who a transaction acts for, using `set_config(..., is_local = true)` so the setting dies with the transaction and never leaks to the next user of a pooled connection.

Context manager	Acts for	Can see	Used by
`workspace_transaction`	a workspace	that workspace's rows; no `users` rows	session lookup, signup, logout
`member_transaction`	workspace + user	above, plus that user's `users` row	`GET /me`, seed
`user_transaction`	a user only	that user's row and memberships	login: find the workspace
`login_transaction`	an email	the one `users` row with that email, read-only	login: find the account

> A forged cookie that pairs a valid token with another workspace's id finds nothing: the lookup runs inside that other workspace's transaction, where RLS hides the real session row. At startup, `database_roles.py` checks that the app role is not a superuser, has no BYPASSRLS, owns nothing in `public` and inherits no other role.

3. Signup, login and logout

Endpoint	Does	Responses
`POST /auth/signup`	Hashes the password, then in one transaction creates the user, a workspace named "Personal", an owner membership and a session.	201, 409 email taken, 422
`POST /auth/login`	Rate-limited password login. Records an audit row.	200, 401, 429 + Retry-After, 422
`POST /auth/logout`	Deletes the session, records an audit row, clears the cookie. Needs the CSRF header.	204, 401, 403
`GET /me`	Returns the user, workspace and CSRF token for the current session.	200, 401
`GET /healthz`, `/readyz`	Liveness; readiness checks Postgres, Redis and MinIO in parallel.	200, 503 with per-check status

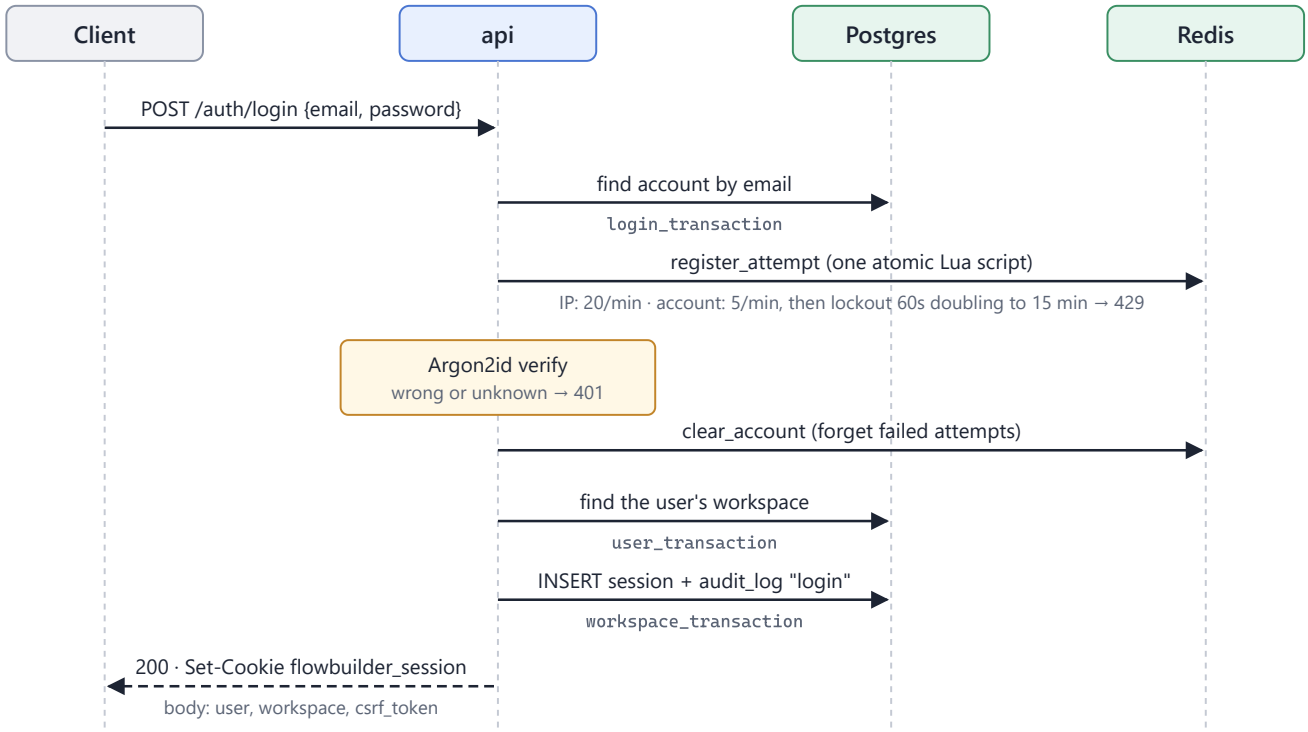


Figure 3. Login. The attempt is counted before the password is checked, so parallel guesses cannot exceed the limit.

Sessions

- Token: 32 random bytes. Only its SHA-256 is stored, so a database copy cannot be replayed as cookies.
- Cookie: HttpOnly, SameSite=Lax, no Domain, 14-day lifetime. Secure is required config, with no default.
- Each session has its own CSRF token, returned in the response body.
- Expiry slides to 14 days from now, at most once an hour.

Hardening details

- Unknown email and wrong password cost the same work (a dummy hash) and return the same 401.
- Argon2id runs in two small thread pools (hash, verify), off the event loop and bounded in memory.
- 422 bodies omit the input, so a rejected password is never echoed.
- Client IP is the socket peer; `X-Forwarded-For` is ignored (`--no-proxy-headers`).
- Logs redact passwords, tokens, bearer headers, URL credentials and `sk-` keys.

4. Data model

Eleven tables across five Alembic migrations. Every table except `users` carries `workspace_id` and is filtered by row-level security. Composite foreign keys include `workspace_id`, so a row can never point at another tenant's row.

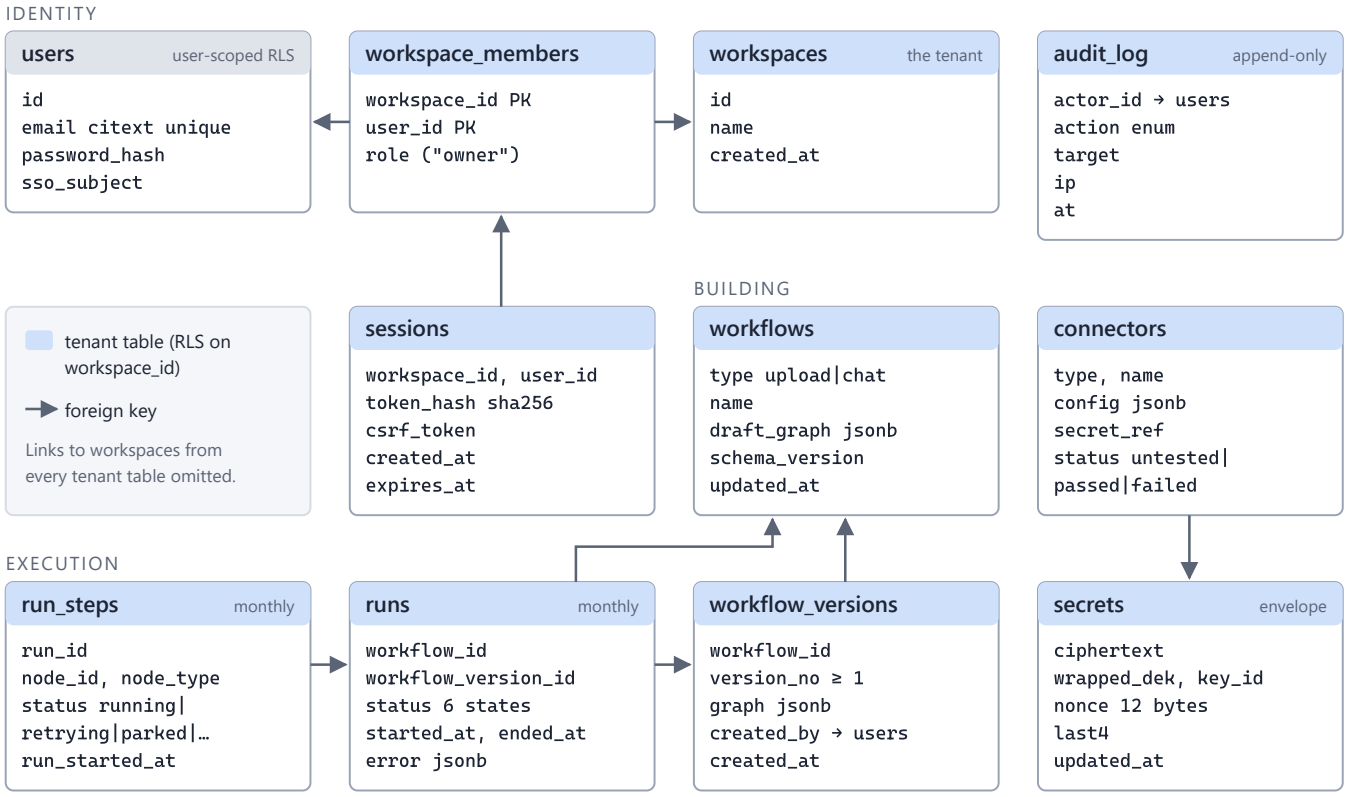


Figure 4. Schema after migration 0005. Workflow, connector, secret and run tables exist but no API uses them yet.

Migration	Adds
`0001_baseline`	`workspaces`, `users`, `workspace_members`
`0002_row_level_security_and_sessions`	RLS helper functions and policies on the baseline tables; `sessions`
`0003_workflows_connectors_secrets_audit`	`workflows`, `workflow_versions`, `secrets`, `connectors`, `audit_log`
`0004_partitioned_runs_and_run_steps`	`runs`, `run_steps`, partitioned by month; `create_run_partitions()`
`0005_user_scoped_row_level_security`	Policies on `users` by user id or login email; members can see their own memberships

5. File map

Backend: `backend/src/app/`

File	What it does
`main.py`	App factory. Wires routers, request logging, the 422 handler that hides input, and the startup role check.
`config.py`	Settings read from the environment: app, migration and bootstrap settings.
`auth.py`	Routes: signup, login, logout, `/me`.
`authentication.py`	Session dependency: cookie → session, CSRF check, sliding refresh. Sets and clears the cookie.
`sessions.py`	Token and CSRF generation, cookie encoding, session SQL.
`tenancy.py`	The four transaction kinds that tell RLS who a transaction acts for.
`passwords.py`	Argon2id hashing in bounded thread pools; constant-cost verify.
`login_rate_limit.py`	Per-IP and per-account limits with doubling lockout, as one Redis Lua script.

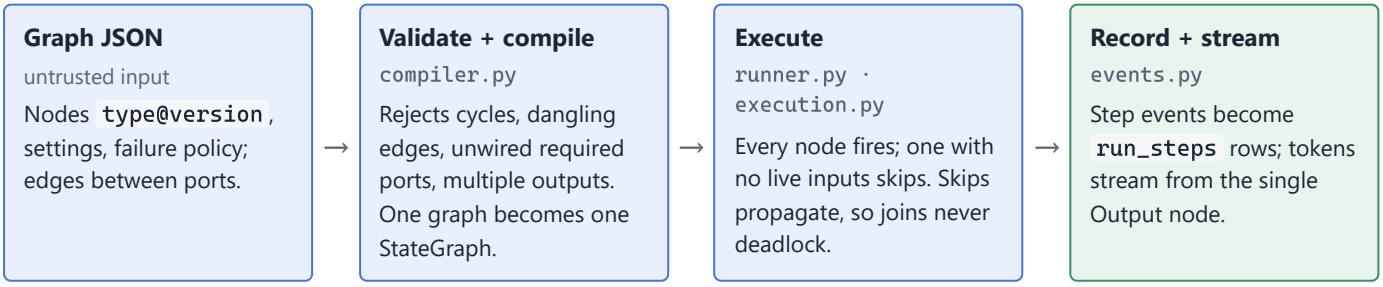
`audit.py`	Audit actions enum and the insert, which runs in the caller's transaction.
`health.py`	`/healthz` and `/readyz` with pluggable checks.
`database.py`	Builds every SQLAlchemy async engine, with bound values hidden from errors.
`database_roles.py`	Role names and the "this role cannot bypass RLS" startup check.
`database_bootstrap.py`	Superuser one-shot: extensions and roles, idempotent, statement logging off.
`run_partitions.py`	Owner one-shot: extends monthly partitions of runs 12 months ahead.
`seed.py`	Demo workspace and a demo user with no password.
`logging_config.py`, `redaction.py`	JSON logs with trace ids; removes secrets from messages and fields.
`telemetry.py`	OpenTelemetry tracer that exports to the collector.
`worker.py`	Celery app on Redis. No tasks yet.

Everything else

Path	What it holds
`backend/migrations/`	Hand-written Alembic migrations 0001–0005.
`backend/tests/`	pytest suites against real Postgres and Redis: auth, login limits, sessions, RLS (workspace and user), cross-tenant probe, route protection, migrations round-trip, redaction, config.
`backend/Dockerfile`	uv-built image with `dev` and `production` stages; runs as a non-root user.
`frontend/src/app/`	Next.js App Router: root layout and a placeholder home page with its Vitest test.
`infra/docker-compose.yml`	The full stack. `.dev.yml` adds live reload via Compose Watch; `.test.yml` is for tests.
`infra/otel-collector/`	Collector image and pipeline config.
`.github/workflows/ci.yml`	Three jobs: backend (ruff, mypy, pytest), frontend (lint, typecheck, test), and the Docker stack coming up healthy.
`postman/`	The API collection and environments.
`spike/`	Throwaway runner prototype on LangGraph, with tests and `FINDINGS.md`.
`docs/runner-contract.md`	Frozen v1 contract for how a saved graph will run.
`tasks/`	`progress.md` (build-plan status) and `lessons.md`.

6. The workflow runner (designed, not yet built)

A workflow is a graph JSON of typed nodes and port-to-port edges. The spike in `spike/runner_spike/` proved the design on LangGraph, and `docs/runner-contract.md` fixes it for the real build in P01_S11.



Secrets and limits travel in a per-run context that is never checkpointed. Cancel uses LangGraph's drain between supersteps.

What comes next (build plan P01)

Status	Subphases
Done	S01 Runner spike · S02 Monorepo, CI, observability · S03 Schema and RLS
In progress	S04 Signup, login and sessions (code merged in PR #11)
Not started	S05 Envelope encryption · S06 Egress guard · S07 Connectors · S08 Node registry · S09 Settings forms · S10 Canvas · S11 Compiler and runner · S12 Acceptance